

DSA Pattern Vault

Pattern-first recall & revision · 24 patterns · Java-primary
keyword → pattern → template

// Keyword → Pattern

Master lookup. Scan the left column for a phrase in your problem, jump to the pattern. Target: match in under 10 seconds.

IF THE PROBLEM SAYS...	REACH FOR	PAGE
sorted array, pair/triplet with target sum, palindrome	Two Pointers	→
longest/shortest contiguous subarray/substring, $\leq K$ distinct	Sliding Window	→
linked-list cycle, find middle, happy number	Fast & Slow Pointers	→
range-sum queries, subarray sum = K, count subarrays	Prefix Sum	→
"seen before", duplicates, complement, group/anagram, frequency	Hashing	→
sorted/rotated, "minimize the maximum", smallest x s.t. feasible	Binary Search	→
next greater/smaller, nearest larger, histogram, temperatures	Monotonic Stack	→
reverse a list / sublist / in k-groups, swap pairs	LinkedList Reversal	→
overlapping intervals, merge/insert interval, meeting rooms	Merge Intervals	→
array holds 1..n, missing/duplicate, first missing positive	Cyclic Sort	→
level order, "by level", minimum depth, right-side view	Tree BFS	→
root-to-leaf path, path sum, height, diameter, LCA	Tree DFS	→
top k , kth largest/smallest, k closest, k most frequent	Heap / Top-K	→
median of a stream, balance two halves	Two Heaps	→
merge k sorted lists, kth smallest in sorted matrix	K-way Merge	→
all subsets/permutations/combinations, N-Queens, generate all	Backtracking	→
shortest/fewest steps (unweighted), grid min moves, spread by layer	Graph BFS	→
connected components, count islands, flood fill, reachability	Graph DFS	→
dependencies, prerequisites, build/course order (DAG)	Topological Sort	→
dynamic connectivity, "are they connected", number of groups	Union-Find	→
prefix search, autocomplete, dictionary, word search II	Trie	→
local rule gives global optimum, interval scheduling, jump/reach	Greedy	→
single number, count bits, power of two, bitmask subsets	Bit Manipulation	→
count the ways, min/max cost, "can you make", longest X	Dynamic Programming	→
weighted graph, non-negative, min cost / time path	Dijkstra	→
negative edges, at most K stops, negative cycle	Bellman-Ford	→
connect all nodes at min cost, spanning tree	Minimum Spanning Tree	→
SCC, bridges, critical connections, articulation points	Tarjan (SCC & Bridges)	→
find a pattern in text, KMP, repeated substring	String Matching	→
answer for range [i,j] from splits, burst balloons	Interval DP	→
design a cache, LRU/LFU, O(1) get/put	LRU / LFU Design	→

> FAST TIE-BREAKERS

When two patterns feel close:

- **Contiguous** best run → Sliding Window; **any** subset/order → DP or Backtracking.
- Shortest path **unweighted** → Graph BFS; components / all-reachable → Graph DFS.
- Need the **median** continuously → Two Heaps; need the **k best** → Heap / Top-K.
- Grouping that only ever **merges** → Union-Find; grouping with **ordering** constraints → Topological Sort.
- "Minimize the maximum / maximize the minimum" almost always → **Binary Search on the answer**.

// Decision Flowchart

How to pick a pattern. Start at the top, answer each branch, land on a pattern. Use this when the keyword table didn't give an instant hit.

```

[diagram]
flowchart TD
    Start([New problem]) --> DS{Main structure?}

    DS -->| "Array / String" | ARR{What is asked?}
    ARR -->|"best contiguous run"| SW[Sliding Window]:::hot
    ARR -->|"sorted · pair-sum · palindrome"| TP[Two Pointers]
    ARR -->|"range sums · subarray = K"| PS[Prefix Sum]
    ARR -->|"seen · dup · complement · group"| HS[Hashing]
    ARR -->|"values are 1..n"| CS[Cyclic Sort]
    ARR -->|"next greater · histogram"| MS[Monotonic Stack]
    ARR -->|"sorted OR minimize-the-max"| BS[Binary Search]:::hot

    DS -->|"Linked List"| LL{Which?}
    LL -->|"cycle · middle"| FS[Fast & Slow Pointers]
    LL -->|"reverse · reorder"| LR[LinkedList Reversal]

    DS -->|"Tree"| TR{Traversal?}
    TR -->|"by level · min depth"| TB[Tree BFS]
    TR -->|"paths · subtree · LCA"| TD[Tree DFS]

    DS -->|"Graph / Grid"| GR{Goal?}
    GR -->|"shortest unweighted"| GB[Graph BFS]:::hot
    GR -->|"components · reachability"| GD[Graph DFS]
    GR -->|"dependency order (DAG)"| TS[Topological Sort]
    GR -->|"dynamic connectivity"| UF[Union-Find]

    DS -->|"Strings + prefixes"| TRIE[Trie]

    Start --> OPT{Optimization / counting?}
    OPT -->|"top-k · kth"| HK[Heap / Top-K]
    OPT -->|"streaming median"| TH[Two Heaps]
    OPT -->|"merge k sorted"| KM[K-way Merge]
    OPT -->|"enumerate ALL configs"| BT[Backtracking]
    OPT -->|"local rule = global opt"| GY[Greedy]
    OPT -->|"overlapping subproblems"| DP[Dynamic Programming]:::hot
    OPT -->|"bit tricks · XOR · mask"| BM[Bit Manipulation]

    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;

```

> THREE QUESTIONS THAT RESOLVE MOST TIES

1. **Is it contiguous?** Yes → Sliding Window / Prefix Sum. No → DP / Backtracking.
2. **Do I need every solution, or just the best/count?** Every → Backtracking. Best/count → Greedy or DP (DP if choices overlap).
3. **Is there a monotonic predicate** (feasible(x) flips once)? Yes → Binary Search on the answer, even when the input isn't "sorted".

// Complexity Cheatsheet

Two tables. (1) Data-structure operation costs, (2) per-pattern Big-O. Memorize the shapes, not the rows.

> DATA-STRUCTURE OPERATIONS

STRUCTURE	ACCESS	SEARCH	INSERT	DELETE	NOTES
Array / ArrayList	$O(1)$	$O(n)$	$O(n)$	$O(n)$	append amortized $O(1)$
Sorted array	$O(1)$	$O(\log n)$	$O(n)$	$O(n)$	binary-searchable
HashMap / HashSet	–	$O(1)^*$	$O(1)^*$	$O(1)^*$	*avg; $O(n)$ worst
Balanced BST / TreeMap	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	ordered iteration
Heap / PriorityQueue	$O(1)$ peek	$O(n)$	$O(\log n)$	$O(\log n)$	pop-min/max $O(\log n)$
Stack / Queue / Deque	$O(1)$ ends	$O(n)$	$O(1)$	$O(1)$	ends only
Linked List	$O(n)$	$O(n)$	$O(1)^\dagger$	$O(1)^\dagger$	† with node ref
Trie	–	$O(L)$	$O(L)$	$O(L)$	L = key length
Union-Find	–	$O(\alpha(n))$	$O(\alpha(n))$	–	inverse Ackermann

* amortized / average case. α = inverse Ackermann ($\approx$ constant).

> PER-PATTERN BIG-O

PATTERN	TIME	SPACE
Two Pointers	$O(n)$ ($+O(n \log n)$ if sorting)	$O(1)$
Sliding Window	$O(n)$	$O(k)$
Fast & Slow Pointers	$O(n)$	$O(1)$
Prefix Sum	$O(n)$	$O(n)$
Hashing	$O(n)$	$O(n)$
Binary Search	$O(\log n) \times \text{feasible}$	$O(1)$
Monotonic Stack	$O(n)$	$O(n)$
LinkedList Reversal	$O(n)$	$O(1)$
Merge Intervals	$O(n \log n)$	$O(n)$
Cyclic Sort	$O(n)$	$O(1)$
Tree BFS	$O(n)$	$O(w)$ width
Tree DFS	$O(n)$	$O(h)$ height
Heap / Top-K	$O(n \log k)$	$O(k)$
Two Heaps	$O(\log n)$ / add	$O(n)$
K-way Merge	$O(N \log k)$	$O(k)$
Backtracking	$O(\text{branch}^{\text{depth}})$	$O(\text{depth})$
Graph BFS	$O(V + E)$	$O(V)$
Graph DFS	$O(V + E)$	$O(V)$
Topological Sort	$O(V + E)$	$O(V + E)$
Union-Find	$O(\alpha(n))$ / op	$O(n)$
Trie	$O(L)$ / op	$O(\text{alphabet} \times \text{chars})$
Greedy	$O(n \log n)$	$O(1)$
Bit Manipulation	$O(1)$ / op, $O(n)$ scan	$O(1)$
Dynamic Programming	$O(\text{states} \times \text{transitions})$	$O(\text{states})$

| Dijkstra | $O(E \log V)$ | $O(V + E)$ | | Bellman-Ford | $O(V \cdot E)$ | $O(V)$ | | Minimum Spanning Tree | $O(E \log E)$ | $O(V)$ | | Tarjan (SCC / Bridges) | $O(V + E)$ | $O(V + E)$ | | String Matching (KMP) | $O(n + m)$ | $O(m)$ | | Interval DP | $O(n^3)$ | $O(n^2)$ | | LRU / LFU Design | $O(1)$ / op | $O(\text{capacity})$ |

> GROWTH REFERENCE

BIG-O	~OPS AT N = 10 ⁵	FEELS LIKE
O(log n)	17	instant
O(n)	10 ⁵	instant
O(n log n)	1.7 × 10 ⁶	fine
O(n ²)	10 ¹⁰	too slow past ~10 ⁴
O(2 ⁿ)	astronomical	only for n ≤ 20 (bitmask/backtracking)

// Two Pointers

> SIGNAL KEYWORDS

sorted array pair/triplet sum palindrome remove/partition in place meet in the middle

> WHEN TO USE / NOT USE

Use on a sorted array (or two sequences) where moving one end predictably raises/lowers a quantity – pair sums, palindromes, in-place partitions.

Not for unsorted "all pairs" (→ Hashing) or best contiguous run (→ Sliding Window).

> DIAGRAM

```
[diagram]
flowchart LR
    I["sorted array · target"] --> S["lo = 0 · hi = n-1"]
    S --> C["a[lo] + a[hi] vs target"]
    C -->|equal| F(["pair found"]):hot
    C -->|"too small"| U["lo++ · need a larger value"]
    C -->|"too large"| D["hi-- · need a smaller value"]
    U --> C
    D --> C
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Two ends squeeze toward the middle.

> TEMPLATE

Java

```
int[] twoSumSorted(int[] a, int target) {
    int lo = 0, hi = a.length - 1;
    while (lo < hi) {
        int sum = a[lo] + a[hi];
        if (sum == target) return new int[]{lo, hi};
        if (sum < target) lo++;    // need bigger → raise low end
        else hi--;                // need smaller → lower high end
    }
    return new int[]{-1, -1};
}
```

Python

```
def two_sum_sorted(a, target):
    lo, hi = 0, len(a) - 1
    while lo < hi:
        s = a[lo] + a[hi]
        if s == target: return (lo, hi)
        if s < target: lo += 1    # need bigger
        else: hi -= 1            # need smaller
    return (-1, -1)
```

C++

```
vector<int> twoSumSorted(vector<int>& a, int target) {
    int lo = 0, hi = a.size() - 1;
    while (lo < hi) {
        int sum = a[lo] + a[hi];
        if (sum == target) return {lo, hi};
        if (sum < target) lo++; // need bigger
        else hi--; // need smaller
    }
    return {-1, -1};
}
```

> COMPLEXITY

Time $O(n)$ to sweep (plus $O(n \log n)$ if you must sort first). **Space** $O(1)$ – just the two indices.

> PITFALLS

- Forgetting the array must be *sorted*.
- Using $lo \leq hi$ when the pair must be distinct.
- Not skipping duplicates in 3Sum.
- Moving the wrong pointer so you overshoot the target.

> CANONICAL PROBLEMS

1. Valid Palindrome **Easy**
2. Two Sum II - Input Array Is Sorted **Medium**
3. Container With Most Water **Medium**
4. 3Sum **Medium**
5. Trapping Rain Water **Hard**

// Sliding Window

> SIGNAL KEYWORDS

longest/shortest contiguous

substring / subarray

at most K distinct

window size k

max/min sum of run

> WHEN TO USE / NOT USE

Use for the *best* contiguous run in a linear structure where a running summary (count, sum, freq map) updates in $O(1)$ as the window moves.

Not for non-contiguous picks (→ DP/backtracking) or when order can be broken (→ sort / two pointers).

> DIAGRAM

```
[diagram]
flowchart LR
    G["grow · sum += nums[right] · right++"] --> V{"window valid?"}
    V -->|"no · too big"| K["shrink · sum -= nums[left] · left++"]
    K --> V
    V -->|"yes"| B["best = max(best, right - left + 1)"]
    B --> G
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
    class B hot;
```

> MNEMONIC

Mnemonic

Grow right, shrink left, track best.

> TEMPLATE

Java

```

int slidingWindow(int[] nums, int k) {
    int left = 0, best = 0, sum = 0;
    for (int right = 0; right < nums.length; right++) {
        sum += nums[right];           // grow by right edge
        while (windowInvalid(sum, k)) { // shrink until valid
            sum -= nums[left];
            left++;
        }
        best = Math.max(best, right - left + 1);
    }
    return best;                     // best valid window length
}

```

Python

```

def sliding_window(nums, k):
    left = best = total = 0
    for right, x in enumerate(nums):
        total += x                # grow
        while window_invalid(total, k): # shrink
            total -= nums[left]; left += 1
        best = max(best, right - left + 1)
    return best

```

C++

```

int slidingWindow(vector<int>& nums, int k) {
    int left = 0, best = 0, sum = 0;
    for (int right = 0; right < nums.size(); ++right) {
        sum += nums[right];           // grow
        while (windowInvalid(sum, k)) { // shrink
            sum -= nums[left++];
        }
        best = max(best, right - left + 1);
    }
    return best;
}

```

> COMPLEXITY

Time $O(n)$ – each index enters and leaves the window once. **Space $O(k)$** – the running summary (freq map $\leq k$ keys); $O(1)$ if a scalar sum.

> PITFALLS

- Shrinking with `if` instead of `while`.
- Forgetting to update the summary on *both* edges.
- Off-by-one on length (`right - left + 1`).
- Fixed-size windows just pop the element leaving at `right - k`.

> CANONICAL PROBLEMS

1. Maximum Average Subarray I **Easy**
2. Longest Substring Without Repeating Characters **Medium**
3. Fruit Into Baskets **Medium**
4. Minimum Size Subarray Sum **Medium**
5. Minimum Window Substring **Hard**

// Fast & Slow Pointers

> SIGNAL KEYWORDS

linked list cycle find middle nth from end cycle entry happy number

> WHEN TO USE / NOT USE

Use on a linked list or implicit functional graph (each node has one "next") to detect a cycle, find its entry, or locate the middle – all in $O(1)$ space.

Not when you need random access or index arithmetic (→ Two Pointers on arrays).

> DIAGRAM

```
[diagram]
flowchart LR
    n1((1)) --> n2((2)) --> n3((3)) --> n4((4)) --> n5((5))
    n5 --> n3
    n3::hot
    cap["slow +1 and fast +2 collide inside the cycle"]::hot
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Tortoise and hare meet in cycles.

> TEMPLATE

Java

```
boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;        // 1 step
        fast = fast.next.next;    // 2 steps
        if (slow == fast) return true; // met → cycle exists
    }
    return false;                // fast fell off → no cycle
}
```

Python

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next        # 1 step
        fast = fast.next.next    # 2 steps
        if slow is fast: return True
    return False
```

C++

```
bool hasCycle(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;        // 1 step
        fast = fast->next->next;    // 2 steps
        if (slow == fast) return true;
    }
    return false;
}
```

> COMPLEXITY

Time $O(n)$ – fast laps slow within one cycle length. **Space** $O(1)$ – two pointers, no hash set.

> PITFALLS

- Null-checking `fast.next` before `fast.next.next`.
- To find the cycle *start*, reset one pointer to head and advance both by 1.
- Middle definition (upper vs lower) depends on the fast start.

> CANONICAL PROBLEMS

1. Middle of the Linked List **Easy**
2. Linked List Cycle **Easy**
3. Happy Number **Easy**
4. Linked List Cycle II **Medium**
5. Find the Duplicate Number **Medium**

// Prefix Sum

> SIGNAL KEYWORDS

range sum query

subarray sum = K

count subarrays

array fixed, many queries

2D region sum

> WHEN TO USE / NOT USE

Use when you need sums (or counts) over many subarrays/ranges of a *fixed* array – a hashmap of prefixes turns "sum == K" into $O(n)$.

Not when values change often (→ Fenwick/segment tree) or you need max/min of a range (→ sparse table / monotonic).

> DIAGRAM

```
[diagram]
flowchart LR
    A["prefix P[i] = P[i-1] + a[i]"] --> B["sum(i..j) = P[j+1] - P[i]"]
    B --> C["count subarrays with sum = k"]
    C --> D["running prefix + hashmap · add seen[prefix - k]"]
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
    class D hot;
```

> MNEMONIC

Mnemonic

Precompute totals; subtract two prefixes.

> TEMPLATE

Java

```
int subarraySum(int[] nums, int k) {
    Map<Integer,Integer> seen = new HashMap<>();
    seen.put(0, 1); // empty prefix seeds the map
    int sum = 0, count = 0;
    for (int x : nums) {
        sum += x; // running prefix
        count += seen.getOrDefault(sum - k, 0); // prefixes that close a window
        seen.merge(sum, 1, Integer::sum);
    }
    return count;
}
```

Python

```
def subarray_sum(nums, k):
    seen = {0: 1} # empty prefix
    total = count = 0
    for x in nums:
        total += x
        count += seen.get(total - k, 0)
        seen[total] = seen.get(total, 0) + 1
    return count
```

C++

```
int subarraySum(vector<int>& nums, int k) {
    unordered_map<int,int> seen{{0, 1}};
    int sum = 0, count = 0;
    for (int x : nums) {
        sum += x;
        count += seen.count(sum - k) ? seen[sum - k] : 0;
        seen[sum]++;
    }
    return count;
}
```

> COMPLEXITY

Time $O(n)$ single pass. **Space** $O(n)$ for the prefix map ($O(1)$ if only a plain prefix array with direct range queries).

> PITFALLS

- Forgetting to seed `seen[0] = 1`.
- Off-by-one between prefix index and array index.
- Integer overflow on large arrays (use long).
- Using a prefix array when values mutate.

> CANONICAL PROBLEMS

1. Running Sum of 1d Array **Easy**
2. Range Sum Query - Immutable **Easy**
3. Subarray Sum Equals K **Medium**
4. Product of Array Except Self **Medium**
5. Contiguous Array **Medium**

// Hashing

> SIGNAL KEYWORDS

seen before

duplicates

complement / two-sum

group by key

frequency count

> WHEN TO USE / NOT USE

Use when you need $O(1)$ membership, frequency, complement, or grouping and order doesn't matter. A `HashMap/HashSet` collapses an $O(n^2)$ scan to $O(n)$.

Not when you need sorted order or range queries ($\rightarrow$ `TreeMap` / sorting) – hashing loses ordering.

> DIAGRAM

```
[diagram]
flowchart LR
    X["for each x"] --> Q{"map has (target - x)?"}
    Q -->|yes| F["return the pair"]:::hot
    Q -->|no| P["map[x] = index"]
    P --> X
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Trade memory for $O(1)$ lookups.

> TEMPLATE

Java

```
int[] twoSum(int[] nums, int target) {
    Map<Integer,Integer> idx = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int need = target - nums[i];
        if (idx.containsKey(need)) // complement already seen
            return new int[]{idx.get(need), i};
        idx.put(nums[i], i); // record value → index
    }
    return new int[]{-1, -1};
}
```

Python

```
def two_sum(nums, target):
    idx = {}
    for i, x in enumerate(nums):
        need = target - x
        if need in idx: # complement seen
            return [idx[need], i]
        idx[x] = i
    return [-1, -1]
```

C++

```
vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int,int> idx;
    for (int i = 0; i < nums.size(); ++i) {
        int need = target - nums[i];
        if (idx.count(need)) return {idx[need], i};
        idx[nums[i]] = i;
    }
    return {-1, -1};
}
```

> COMPLEXITY

Time $O(n)$ average (hash ops $O(1)$ amortized). **Space** $O(n)$ for the map/set.

> PITFALLS

- Inserting *before* checking the complement (matches an element with itself).
- Relying on iteration order.
- Hashing mutable objects.
- Worst-case $O(n)$ buckets on adversarial keys.

> CANONICAL PROBLEMS

1. Two Sum **Easy**
2. Contains Duplicate **Easy**
3. Valid Anagram **Easy**
4. Group Anagrams **Medium**
5. Longest Consecutive Sequence **Medium**

// Binary Search (+ on answer)

> SIGNAL KEYWORDS

sorted / rotated

minimize the maximum

smallest x such that...

monotonic predicate

kth / threshold

> WHEN TO USE / NOT USE

Use on a sorted array, or on an *answer space* where `feasible(x)` flips false→true monotonically ("binary search on the answer").

Not when no monotone predicate exists – you can't bisect what isn't ordered.

> DIAGRAM

```
[diagram]
flowchart LR
    A["F F F F | T T T T · monotonic predicate"] --> M["mid = lo + (hi - lo) / 2"]
    M --> Q{"feasible(mid)?"}
    Q -->|true| L["hi = mid · keep, go left"]:::hot
    Q -->|false| R["lo = mid + 1 · go right"]
    L --> M
    R --> M
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Halve the space; keep feasible side.

> TEMPLATE

Java

```
int firstTrue(int lo, int hi) {           // search answer space [lo, hi]
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;    // avoid overflow
        if (feasible(mid)) hi = mid;     // works → keep it, search left
        else lo = mid + 1;               // fails → search right
    }
    return lo;                           // smallest feasible value
}
```

Python

```
def first_true(lo, hi):                  # answer space [lo, hi]
    while lo < hi:
        mid = (lo + hi) // 2
        if feasible(mid): hi = mid      # keep, go left
        else: lo = mid + 1              # go right
    return lo
```

C++

```
int firstTrue(int lo, int hi) {          // answer space [lo, hi]
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;    // avoid overflow
        if (feasible(mid)) hi = mid;     // keep, go left
        else lo = mid + 1;               // go right
    }
    return lo;
}
```

> COMPLEXITY

Time $O(\log n)$ iterations × cost of feasible. Space $O(1)$.

> PITFALLS

- $(lo+hi)/2$ overflow (use $lo + (hi-lo)/2$).
- $lo = mid$ causing an infinite loop – pair $hi = mid$ with $lo = mid + 1$.
- Confusing *first-true* vs *last-true* boundary.
- Wrong initial hi (must be inclusive of a valid answer).

> CANONICAL PROBLEMS

1. Binary Search **Easy**
2. First Bad Version **Easy**
3. Find Minimum in Rotated Sorted Array **Medium**
4. Koko Eating Bananas **Medium**

// Monotonic Stack

> SIGNAL KEYWORDS

next greater/smaller

nearest larger

histogram

span / temperatures

pop while ...

> WHEN TO USE / NOT USE

Use to find, for every element, the nearest greater/smaller element (or the range it dominates) in one linear pass – keep a stack whose values stay monotonic.

Not when you need a global sort or random access; the stack only knows its current run.

> DIAGRAM

```
[diagram]
flowchart LR
    N["nums[i] arrives"] --> C{"nums[i] beats stack top?"}
    C -->|yes| P["pop · i is its next-greater"]:::hot
    P --> C
    C -->|no| U["push i · stack stays decreasing"]
    U --> N
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Pop losers; stack stays sorted.

> TEMPLATE

Java

```
int[] nextGreater(int[] nums) {
    int n = nums.length;
    int[] res = new int[n];
    Arrays.fill(res, -1);
    Deque<Integer> st = new ArrayDeque<>(); // indices, values decreasing
    for (int i = 0; i < n; i++) {
        while (!st.isEmpty() && nums[i] > nums[st.peek()])
            res[st.pop()] = nums[i]; // i answers the popped index
        st.push(i);
    }
    return res; // leftovers stay -1
}
```

Python

```
def next_greater(nums):
    res = [-1] * len(nums)
    st = [] # indices, decreasing values
    for i, x in enumerate(nums):
        while st and x > nums[st[-1]]:
            res[st.pop()] = x # i answers popped index
        st.append(i)
    return res
```

C++

```
vector<int> nextGreater(vector<int>& nums) {
    int n = nums.size();
    vector<int> res(n, -1);
    stack<int> st; // indices, decreasing values
    for (int i = 0; i < n; ++i) {
        while (!st.empty() && nums[i] > nums[st.top()]) {
            res[st.top()] = nums[i]; st.pop();
        }
        st.push(i);
    }
    return res;
}
```

> COMPLEXITY

Time $O(n)$ – each index is pushed and popped at most once. **Space** $O(n)$ for the stack.

> PITFALLS

- Storing values when you need indices (or vice versa).
- Wrong strictness ($>$ vs $>=$) with duplicates.
- Picking the wrong monotonic direction (increasing vs decreasing) for "greater" vs "smaller".
- Forgetting unresolved elements keep the default.

> CANONICAL PROBLEMS

1. Next Greater Element I **Easy**
2. Daily Temperatures **Medium**
3. Online Stock Span **Medium**
4. Sum of Subarray Minimums **Medium**
5. Largest Rectangle in Histogram **Hard**

// LinkedList Reversal

> SIGNAL KEYWORDS

reverse a list

reverse sublist

k-group

swap pairs

reorder / palindrome

> WHEN TO USE / NOT USE

Use to reverse all or part of a singly linked list in place by flipping next pointers – $O(1)$ extra space.

Not when you need repeated backward traversal or random access ($\rightarrow$ array / deque).

> DIAGRAM

```
[diagram]
flowchart LR
    subgraph B["before"]
        a1((1)) --> a2((2)) --> a3((3)) --> an(("null"))
    end
    M["nxt = cur.next · cur.next = prev · prev = cur · cur = nxt"]:::hot
    subgraph A["after"]
        b3((3)) --> b2((2)) --> b1((1)) --> bn(("null"))
    end
    B --> M --> A
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Rewire next backward, one hop each.

> TEMPLATE

Java

```

ListNode reverse(ListNode head) {
    ListNode prev = null, cur = head;
    while (cur != null) {
        ListNode next = cur.next; // save the rest first
        cur.next = prev;          // flip pointer backward
        prev = cur;               // advance prev
        cur = next;               // advance cur
    }
    return prev;                  // prev is the new head
}

```

Python

```

def reverse(head):
    prev, cur = None, head
    while cur:
        nxt = cur.next      # save rest
        cur.next = prev     # flip
        prev, cur = cur, nxt # advance both
    return prev

```

C++

```

ListNode* reverse(ListNode* head) {
    ListNode *prev = nullptr, *cur = head;
    while (cur) {
        ListNode* next = cur->next; // save rest
        cur->next = prev;           // flip
        prev = cur; cur = next;    // advance
    }
    return prev;
}

```

> COMPLEXITY

Time $O(n)$ single pass. **Space** $O(1)$ – three pointers, no copy.

> PITFALLS

- Not saving `cur.next` before overwriting it (loses the rest).
- Returning `cur` (null) instead of `prev`.
- Sublist reversal needs a dummy head and re-linking both boundaries.
- Off-by-one when counting to position m/n .

> CANONICAL PROBLEMS

1. Reverse Linked List **Easy**
2. Swap Nodes in Pairs **Medium**
3. Reverse Linked List II **Medium**
4. Rotate List **Medium**
5. Reverse Nodes in k-Group **Hard**

// Merge Intervals

> SIGNAL KEYWORDS

overlapping ranges

merge intervals

insert interval

meeting rooms

conflicts / free time

> WHEN TO USE / NOT USE

Use when the input is a set of intervals and the answer depends on overlaps – sort by start, then sweep left to right merging or counting.

Not when data has no natural ordering to sort on, or overlap isn't the question.

> DIAGRAM

```
[diagram]
flowchart LR
    S["sort intervals by start"] --> I["scan each interval"]
    I --> Q{"start ≤ last.end?"}
    Q -->|"yes · overlap"| M["last.end = max(last.end, end)"]:::hot
    Q -->|"no · gap"| A["append as a new interval"]
    M --> I
    A --> I
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Sort by start; extend or append.

> TEMPLATE

Java

```
int[][] merge(int[][] iv) {
    Arrays.sort(iv, (a, b) -> Integer.compare(a[0], b[0])); // by start
    List<int[]> out = new ArrayList<>();
    for (int[] cur : iv) {
        int n = out.size();
        if (n > 0 && cur[0] <= out.get(n - 1)[1]) // overlap?
            out.get(n - 1)[1] = Math.max(out.get(n - 1)[1], cur[1]);
        else
            out.add(cur); // gap -> new
    }
    return out.toArray(new int[0][]);
}
```

Python

```
def merge(iv):
    iv.sort(key=lambda x: x[0]) # by start
    out = []
    for s, e in iv:
        if out and s <= out[-1][1]: # overlap
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e]) # gap
    return out
```

C++

```
vector<vector<int>> merge(vector<vector<int>>& iv) {
    sort(iv.begin(), iv.end()); // by start
    vector<vector<int>> out;
    for (auto& cur : iv) {
        if (!out.empty() && cur[0] <= out.back()[1])
            out.back()[1] = max(out.back()[1], cur[1]);
        else out.push_back(cur);
    }
    return out;
}
```

> COMPLEXITY

Time $O(n \log n)$ dominated by the sort. **Space** $O(n)$ for the output ($O(\log n)$ if sorting in place and counting only).

> PITFALLS

- Forgetting to sort first.
- Comparator overflow ($a[0]-b[0]$ on large ints – use `Integer.compare`).
- Touching intervals $[1,2], [2,3]$ – decide $\leq$ vs $<$.

- Extending to `max(last.end, cur.end)`, not just `cur.end`.

> CANONICAL PROBLEMS

1. Meeting Rooms **Easy**
2. Merge Intervals **Medium**
3. Insert Interval **Medium**
4. Non-overlapping Intervals **Medium**
5. Meeting Rooms II **Medium**

// Cyclic Sort

> SIGNAL KEYWORDS

values in 1..n missing number duplicate number first missing positive range known

> WHEN TO USE / NOT USE

Use when an array of length `n` holds numbers from a contiguous range (`1..n` or `0..n-1`): place each value at its home index, then a scan finds what's missing or duplicated – $O(n)$ time, $O(1)$ space.

Not for arbitrary or sparse value ranges.

> DIAGRAM

```
[diagram]
flowchart LR
    I["i = 0"] --> Q{"a[i] already at its home index?"}
    Q -->|no| S["swap a[i] → index a[i]-1"]
    S --> Q
    Q -->|yes| N["i++"]
    N --> I
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Send each number to its index.

> TEMPLATE

Java

```
void cyclicSort(int[] a) {
    int i = 0;
    while (i < a.length) {
        int home = a[i] - 1; // value v belongs at index v-1
        if (a[i] != a[home]) { // not home AND not a dup there
            int t = a[i]; a[i] = a[home]; a[home] = t; // swap it home
        } else {
            i++; // already placed -> advance
        }
    }
}
```

Python

```
def cyclic_sort(a):
    i = 0
    while i < len(a):
        home = a[i] - 1 # v belongs at v-1
        if a[i] != a[home]: # guard against dup loop
            a[i], a[home] = a[home], a[i]
        else:
            i += 1
```

C++

```
void cyclicSort(vector<int>& a) {
    int i = 0;
    while (i < a.size()) {
        int home = a[i] - 1;          // v belongs at v-1
        if (a[i] != a[home]) swap(a[i], a[home]);
        else i++;
    }
}
```

> COMPLEXITY

Time $O(n)$ – each swap puts one value home permanently. **Space** $O(1)$ in place.

> PITFALLS

- Comparing to index i instead of $a[home]$ (duplicates cause an infinite loop – the $a[i] != a[home]$ guard is essential).
- Base offset ($0..n-1$ vs $1..n$).
- Values outside range must be ignored, not indexed.

> CANONICAL PROBLEMS

1. Missing Number **Easy**
2. Set Mismatch **Easy**
3. Find All Numbers Disappeared in an Array **Easy**
4. Find the Duplicate Number **Medium**
5. First Missing Positive **Hard**

// Tree BFS

> SIGNAL KEYWORDS

level order by level minimum depth right side view zigzag / connect

> WHEN TO USE / NOT USE

Use to process a tree level by level – anything that groups nodes by depth, or needs the shallowest result (minimum depth, first level to satisfy X).

Not for root-to-leaf paths or subtree aggregation (→ Tree DFS).

> DIAGRAM

```
[diagram]
flowchart TB
    A((1)) --> B((2))
    A((1)) --> C((3))
    B --> D((4))
    B --> E((5))
    C --> F((6))
    L["queue drains one level at a time · [1] → [2,3] → [4,5,6]"]::hot
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Queue a level; drain it fully.

> TEMPLATE

Java

```

List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();
    Queue<TreeNode> q = new LinkedList<>();
    if (root != null) q.add(root);
    while (!q.isEmpty()) {
        int n = q.size();           // freeze this level's width
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            TreeNode node = q.poll();
            level.add(node.val);
            if (node.left != null) q.add(node.left);
            if (node.right != null) q.add(node.right);
        }
        res.add(level);
    }
    return res;
}

```

Python

```

from collections import deque
def level_order(root):
    res, q = [], deque([root] if root else [])
    while q:
        level = []
        for _ in range(len(q)):      # freeze width
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        res.append(level)
    return res

```

C++

```

vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> res; queue<TreeNode*> q;
    if (root) q.push(root);
    while (!q.empty()) {
        int n = q.size(); vector<int> level;
        for (int i = 0; i < n; i++) {
            TreeNode* node = q.front(); q.pop();
            level.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        res.push_back(level);
    }
    return res;
}

```

> COMPLEXITY

Time $O(n)$ – each node enqueued/dequeued once. **Space $O(w)$** – the widest level.

> PITFALLS

- Not snapshotting `q.size()` before the inner loop (mixes levels).
- Enqueuing null children.
- Forgetting the empty-root guard.
- Reaching for DFS when the question is inherently level-based.

> CANONICAL PROBLEMS

1. Average of Levels in Binary Tree **Easy**
2. Minimum Depth of Binary Tree **Easy**
3. Binary Tree Level Order Traversal **Medium**

4. Binary Tree Right Side View **Medium**
5. Binary Tree Zigzag Level Order Traversal **Medium**

// Tree DFS

> SIGNAL KEYWORDS

root-to-leaf path path sum height / diameter subtree property lowest common ancestor

> WHEN TO USE / NOT USE

Use to explore paths or fold subtree values – recurse to children, then combine their results at each node. Ideal for path sums, height, diameter, LCA.

Not for shortest unweighted depth or per-level grouping (→ Tree BFS).

> DIAGRAM

```
[diagram]
flowchart TB
    A((1)) --> B((2))
    A((1)) --> C((3))
    B --> D((4))
    B --> E((5))
    O["preorder dive · 1 → 2 → 4 → 5 → 3 · combine on the way up"]:::hot
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Recurse down; combine coming back up.

> TEMPLATE

Java

```
boolean hasPathSum(TreeNode node, int target) {
    if (node == null) return false;
    target -= node.val;           // consume this node
    if (node.left == null && node.right == null)
        return target == 0;      // leaf: exact hit?
    return hasPathSum(node.left, target)    // try left subtree
        || hasPathSum(node.right, target);  // or right subtree
}
```

Python

```
def has_path_sum(node, target):
    if not node: return False
    target -= node.val           # consume node
    if not node.left and not node.right:
        return target == 0      # leaf hit?
    return (has_path_sum(node.left, target)
            or has_path_sum(node.right, target))
```

C++

```
bool hasPathSum(TreeNode* node, int target) {
    if (!node) return false;
    target -= node->val;         // consume node
    if (!node->left && !node->right)
        return target == 0;     // leaf hit?
    return hasPathSum(node->left, target)
        || hasPathSum(node->right, target);
}
```

> COMPLEXITY

Time $O(n)$ – visit each node once. **Space** $O(h)$ – recursion stack, h = tree height ($O(n)$ if skewed).

> PITFALLS

- Confusing "null" with "leaf" (a leaf has both children null).
- Missing the base case.
- Sharing mutable path state across branches without undoing it (backtrack).
- Stack overflow on degenerate skewed trees.

> CANONICAL PROBLEMS

1. Maximum Depth of Binary Tree **Easy**
2. Path Sum **Easy**
3. Diameter of Binary Tree **Easy**
4. Lowest Common Ancestor of a Binary Tree **Medium**
5. Binary Tree Maximum Path Sum **Hard**

// Heap / Top-K

> SIGNAL KEYWORDS

top k kth largest/smallest k closest k most frequent running best

> WHEN TO USE / NOT USE

Use to select the k best items without sorting everything – a size-k heap holds the current winners in $O(n \log k)$.

Not when you need a full ordering (just sort) or a rolling median (→ Two Heaps).

> DIAGRAM

```
[diagram]
flowchart LR
    X["push each value into a size-k min-heap"] --> Q{"heap size > k?"}
    Q -->|yes| P["pop the smallest · drop it"]
    Q -->|no| K["keep"]
    P --> R["root = kth largest"]:::hot
    K --> R
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Keep K inside a size-K heap.

> TEMPLATE

Java

```
int kthLargest(int[] nums, int k) {
    PriorityQueue<Integer> heap = new PriorityQueue<>(); // min-heap
    for (int x : nums) {
        heap.offer(x);
        if (heap.size() > k) heap.poll(); // evict smallest -> keep top k
    }
    return heap.peek(); // root = kth largest
}
```

Python

```
import heapq
def kth_largest(nums, k):
    heap = [] # min-heap
    for x in nums:
        heapq.heappush(heap, x)
        if len(heap) > k:
            heapq.heappop(heap) # drop smallest
    return heap[0] # kth largest
```

C++

```
int kthLargest(vector<int>& nums, int k) {
    priority_queue<int, vector<int>, greater<int>> heap; // min-heap
    for (int x : nums) {
        heap.push(x);
        if (heap.size() > k) heap.pop(); // drop smallest
    }
    return heap.top();
}
```

> COMPLEXITY

Time $O(n \log k)$ – n pushes, each $O(\log k)$. **Space** $O(k)$ for the heap.

> PITFALLS

- Using a full-size heap (that's $O(n \log n)$).
- Wrong polarity – a *min*-heap keeps the k *largest*.
- Forgetting to evict once size exceeds k .
- Unstable ties when a comparator ignores a tiebreak.

> CANONICAL PROBLEMS

1. Kth Largest Element in a Stream **Easy**
2. Last Stone Weight **Easy**
3. K Closest Points to Origin **Medium**
4. Top K Frequent Elements **Medium**
5. Kth Largest Element in an Array **Medium**

// Two Heaps

> SIGNAL KEYWORDS

median of a stream

balance two halves

sliding window median

two criteria

running middle

> WHEN TO USE / NOT USE

Use when you must repeatedly read the middle (or a balance point) of a growing set – a max-heap of the low half and a min-heap of the high half give $O(1)$ median, $O(\log n)$ inserts.

Not when a single heap suffices ($\rightarrow$ Heap / Top-K).

> DIAGRAM

```
[diagram]
flowchart LR
    L["max-heap · low half"]:::hot --- H["min-heap · high half"]
    ADD["add x → push low, shift max to high, rebalance"] --> L
    MED(["median = top of the larger heap"]):::hot
    L --- MED
    H --- MED
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Max-heap low half, min-heap high.

> TEMPLATE

Java

```

PriorityQueue<Integer> lo = new PriorityQueue<>(Collections.reverseOrder()); // max
PriorityQueue<Integer> hi = new PriorityQueue<>(); // min
void add(int x) {
    lo.offer(x); // 1. always add to low half
    hi.offer(lo.poll()); // 2. move its max up to high half
    if (hi.size() > lo.size()) // 3. keep lo >= hi in size
        lo.offer(hi.poll());
}
double median() {
    return lo.size() > hi.size()
        ? lo.peek() : (lo.peek() + hi.peek()) / 2.0;
}

```

Python

```

import heapq
lo, hi = [], [] # lo: max-heap (negated), hi: min-heap
def add(x):
    heapq.heappush(lo, -heapq.heappushpop(hi, x)) # x -> hi, hi.min -> lo
    if len(lo) > len(hi): # rebalance
        heapq.heappush(hi, -heapq.heappop(lo))
def median():
    return -lo[0] if len(lo) < len(hi) else (hi[0] - lo[0]) / 2

```

C++

```

priority_queue<int> lo; // max-heap
priority_queue<int, vector<int>, greater<int>> hi; // min-heap
void add(int x) {
    lo.push(x);
    hi.push(lo.top()); lo.pop(); // shift max up
    if (hi.size() > lo.size()) { lo.push(hi.top()); hi.pop(); }
}
double median() {
    return lo.size() > hi.size() ? lo.top() : (lo.top() + hi.top()) / 2.0;
}

```

> COMPLEXITY

Time $O(\log n)$ per insertion, $O(1)$ per median read. Space $O(n)$ across both heaps.

> PITFALLS

- Letting the two sizes drift (enforce $lo.size == hi.size$ or $lo + 1$).
- Integer division on an even-count median.
- Skipping the "push-then-shift" step that keeps every low element $\leq$ every high element.

> CANONICAL PROBLEMS

1. Total Cost to Hire K Workers **Medium**
2. Find Median from Data Stream **Hard**
3. IPO **Hard**
4. Sliding Window Median **Hard**
5. Minimize Deviation in Array **Hard**

// K-way Merge

> SIGNAL KEYWORDS

merge k sorted lists kth smallest in matrix smallest range k pairs smallest sum merge k streams

> WHEN TO USE / NOT USE

Use to combine k already-sorted sequences: a min-heap of the current head from each list always yields the global next-smallest.

Not when inputs aren't sorted (sort or use a different structure first).

> DIAGRAM

```
[diagram]
flowchart LR
    H["min-heap of the current head of each of k lists"] --> P["pop smallest → output"]:::hot
    P --> N["push the popped node's next"]
    N --> H
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Heap the k heads; pop smallest.

> TEMPLATE

Java

```
ListNode mergeK(ListNode[] lists) {
    PriorityQueue<ListNode> pq = new PriorityQueue<>((a, b) -> a.val - b.val);
    for (ListNode l : lists) if (l != null) pq.offer(l); // seed with heads
    ListNode dummy = new ListNode(0), tail = dummy;
    while (!pq.isEmpty()) {
        ListNode node = pq.poll(); // smallest current head
        tail.next = node; tail = node;
        if (node.next != null) pq.offer(node.next); // push its successor
    }
    return dummy.next;
}
```

Python

```
import heapq
def merge_k(lists):
    pq = [(l.val, i, l) for i, l in enumerate(lists) if l] # (val, tiebreak, node)
    heapq.heapify(pq)
    dummy = tail = ListNode(0)
    while pq:
        _, i, node = heapq.heappop(pq)
        tail.next = node; tail = node
        if node.next: heapq.heappush(pq, (node.next.val, i, node.next))
    return dummy.next
```

C++

```
ListNode* mergeK(vector<ListNode*>& lists) {
    auto cmp = [](ListNode* a, ListNode* b){ return a->val > b->val; };
    priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> pq(cmp);
    for (auto l : lists) if (l) pq.push(l);
    ListNode dummy(0), *tail = &dummy;
    while (!pq.empty()) {
        ListNode* node = pq.top(); pq.pop();
        tail->next = node; tail = node;
        if (node->next) pq.push(node->next);
    }
    return dummy.next;
}
```

> COMPLEXITY

Time $O(N \log k)$ – N total elements, heap of size $\leq k$. **Space** $O(k)$ for the heap.

> PITFALLS

- Seeding null lists into the heap.
- Comparator overflow on $a.val - b.val$.
- Forgetting to push `node.next` after popping.
- A Python heap needs a tiebreak index so it never compares raw nodes.

> CANONICAL PROBLEMS

1. Merge Two Sorted Lists **Easy**
2. Kth Smallest Element in a Sorted Matrix **Medium**
3. Find K Pairs with Smallest Sums **Medium**
4. Merge k Sorted Lists **Hard**
5. Smallest Range Covering Elements from K Lists **Hard**

// Backtracking

> SIGNAL KEYWORDS

all subsets / permutations

combinations to target

generate all ways

N-Queens / sudoku

word search

> WHEN TO USE / NOT USE

Use to enumerate every valid configuration by building a candidate incrementally and undoing each choice on the way back.

Not when you only need a count or an optimum with overlapping subproblems (→ DP) – backtracking is exponential.

> DIAGRAM

```
[diagram]
flowchart TD
    S["state · partial candidate"] --> C["choose an option"]
    C --> E["explore · recurse deeper"]
    E --> G{"complete?"}
    G -->|yes| R["record a solution"]:::hot
    E --> U["un-choose · undo the option"]:::hot
    U --> C
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Choose, explore, un-choose.

> TEMPLATE

Java

```
void backtrack(int[] nums, int start, List<Integer> path,
               List<List<Integer>> res) {
    res.add(new ArrayList<>(path)); // record a COPY of current path
    for (int i = start; i < nums.length; i++) {
        path.add(nums[i]); // choose
        backtrack(nums, i + 1, path, res); // explore (i+1 = no reuse)
        path.remove(path.size() - 1); // un-choose (backtrack)
    }
}
```

Python

```
def backtrack(nums, start, path, res):
    res.append(path[:]) # copy current path
    for i in range(start, len(nums)):
        path.append(nums[i]) # choose
        backtrack(nums, i + 1, path, res) # explore
        path.pop() # un-choose
```

C++

```
void backtrack(vector<int>& nums, int start,
              vector<int>& path, vector<vector<int>>& res) {
    res.push_back(path);           // copy current path
    for (int i = start; i < nums.size(); i++) {
        path.push_back(nums[i]);   // choose
        backtrack(nums, i + 1, path, res); // explore
        path.pop_back();           // un-choose
    }
}
```

> COMPLEXITY

Time $O(\text{branch}^{\text{depth}})$ – exponential in the search tree. **Space** $O(\text{depth})$ for the recursion path (plus the output).

> PITFALLS

- Storing a reference to path instead of a copy (every result mutates together).
- Wrong start index (i for reuse, i+1 for no reuse).
- Not pruning invalid branches early.
- Forgetting to undo the choice after recursing.

> CANONICAL PROBLEMS

1. Letter Case Permutation **Easy**
2. Subsets **Medium**
3. Permutations **Medium**
4. Combination Sum **Medium**
5. N-Queens **Hard**

// Graph BFS

> SIGNAL KEYWORDS

shortest / fewest steps

unweighted

minimum moves in a grid

spread by layer

nearest

> WHEN TO USE / NOT USE

Use for shortest path or minimum steps in an *unweighted* graph or grid – BFS expands in distance layers, so the first time you reach a node is the shortest.

Not for weighted edges (→ Dijkstra) or enumerating all paths (→ DFS).

> DIAGRAM

```
[diagram]
flowchart TD
    S(("start"))::hot --> L1["layer 1 · 4-dir neighbors"]
    L1 --> L2["layer 2"]
    L2 --> L3["layer 3 ..."]
    L3 --> G(["first reach of target = shortest distance"])::hot
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Queue neighbors; nearest layers first.

> TEMPLATE

Java

```

int bfs(int[][] grid, int[] start, int[] end) {
    int m = grid.length, n = grid[0].length;
    boolean[][] seen = new boolean[m][n];
    Queue<int[]> q = new LinkedList<>();
    q.add(start); seen[start[0]][start[1]] = true;
    int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
    for (int steps = 0; !q.isEmpty(); steps++) {
        for (int s = q.size(); s > 0; s--) { // one layer
            int[] c = q.poll();
            if (c[0]==end[0] && c[1]==end[1]) return steps;
            for (int[] d : dirs) {
                int r = c[0]+d[0], k = c[1]+d[1];
                if (r<0||r>=m||k<0||k>=n||seen[r][k]||grid[r][k]==1) continue;
                seen[r][k] = true; q.add(new int[]{r, k}); // mark on enqueue
            }
        }
    }
    return -1;
}

```

Python

```

from collections import deque
def bfs(grid, start, end):
    m, n = len(grid), len(grid[0])
    seen = {start}; q = deque([start]); steps = 0
    while q:
        for _ in range(len(q)): # one layer
            r, c = q.popleft()
            if (r, c) == end: return steps
            for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
                nr, nc = r+dr, c+dc
                if 0<=nr<m and 0<=nc<n and (nr,nc) not in seen and grid[nr][nc]==0:
                    seen.add((nr,nc)); q.append((nr,nc))
        steps += 1
    return -1

```

C++

```

int bfs(vector<vector<int>>& g, pair<int,int> s, pair<int,int> e) {
    int m = g.size(), n = g[0].size();
    vector<vector<bool>> seen(m, vector<bool>(n));
    queue<pair<int,int>> q; q.push(s); seen[s.first][s.second] = true;
    int dirs[4][2] = {{1,0},{-1,0},{0,1},{0,-1}};
    for (int steps = 0; !q.empty(); steps++)
        for (int sz = q.size(); sz > 0; sz--) {
            auto [r, c] = q.front(); q.pop();
            if (r==e.first && c==e.second) return steps;
            for (auto& d : dirs) {
                int nr=r+d[0], nc=c+d[1];
                if (nr<0||nr>=m||nc<0||nc>=n||seen[nr][nc]||g[nr][nc]) continue;
                seen[nr][nc]=true; q.push({nr,nc});
            }
        }
    return -1;
}

```

> COMPLEXITY

Time $O(V + E)$ – every node and edge once (grid: $O(m \cdot n)$). **Space** $O(V)$ for the queue and visited set.

> PITFALLS

- Marking visited on *dequeue* instead of *enqueue* (nodes get queued many times).
- Missing bounds checks.
- Not iterating a full layer per step when you need distances.
- Using BFS on weighted edges.

> CANONICAL PROBLEMS

1. Flood Fill **Easy**
2. Number of Islands **Medium**
3. Rotting Oranges **Medium**
4. 01 Matrix **Medium**
5. Word Ladder **Hard**

// Graph DFS

> SIGNAL KEYWORDS

connected components count regions / islands flood fill cycle detection reachability

> WHEN TO USE / NOT USE

Use to explore full reachability – count components, measure regions, detect cycles, or clone a graph. Recurse (or use an explicit stack), marking nodes visited.

Not for shortest path in an unweighted graph (→ Graph BFS).

> DIAGRAM

```
[diagram]
flowchart TD
    V["visit cell / node"] --> M["mark visited"]:::hot
    M --> R["recurse into each unvisited neighbor"]
    R --> V
    R --> B["dead end → backtrack"]
    M --> N(["one full dive = one component"]):::hot
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Dive deep; mark, recurse, backtrack.

> TEMPLATE

Java

```
void dfs(char[][] g, int r, int c) {
    if (r<0||r>=g.length||c<0||c>=g[0].length||g[r][c]!='1') return;
    g[r][c] = '0'; // mark visited (sink land)
    dfs(g, r+1, c); dfs(g, r-1, c);
    dfs(g, r, c+1); dfs(g, r, c-1); // 4 neighbors
}

int numIslands(char[][] g) {
    int count = 0;
    for (int r = 0; r < g.length; r++)
        for (int c = 0; c < g[0].length; c++)
            if (g[r][c] == '1') { count++; dfs(g, r, c); } // new component
    return count;
}
```

Python

```
def num_islands(g):
    def dfs(r, c):
        if not (0<=r<len(g) and 0<=c<len(g[0])) or g[r][c] != '1': return
        g[r][c] = '0' # mark
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            dfs(r+dr, c+dc)
    count = 0
    for r in range(len(g)):
        for c in range(len(g[0])):
            if g[r][c] == '1':
                count += 1; dfs(r, c) # new component
    return count
```

C++

```
void dfs(vector<vector<char>>& g, int r, int c) {
    if (r<0||r>=g.size()||c<0||c>=g[0].size()||g[r][c]!='1') return;
    g[r][c] = '0'; // mark
    dfs(g,r+1,c); dfs(g,r-1,c); dfs(g,r,c+1); dfs(g,r,c-1);
}
int numIslands(vector<vector<char>>& g) {
    int count = 0;
    for (int r=0;r<g.size();r++)
        for (int c=0;c<g[0].size();c++)
            if (g[r][c]=='1') { count++; dfs(g,r,c); }
    return count;
}
```

> COMPLEXITY

Time $O(V + E)$ – each node/edge once (grid: $O(m \cdot n)$). **Space** $O(V)$ for the recursion stack / visited set.

> PITFALLS

- Not marking visited (infinite recursion).
- Marking too late so neighbors re-enter.
- Stack overflow on very large grids (switch to an explicit stack).
- Directed-cycle detection needs an on-stack / 3-color state, not just "visited".

> CANONICAL PROBLEMS

1. Flood Fill **Easy**
2. Number of Islands **Medium**
3. Max Area of Island **Medium**
4. Clone Graph **Medium**
5. Course Schedule **Medium**

// Topological Sort

> SIGNAL KEYWORDS

dependencies / prerequisites

build order

course schedule

DAG ordering

can finish?

> WHEN TO USE / NOT USE

Use to linearly order the nodes of a **DAG** so every edge points forward – task scheduling with prerequisites. Kahn's BFS also detects cycles (not all nodes emitted $\Rightarrow$ cycle).

Not on undirected graphs or where cycles are allowed.

> DIAGRAM

```
[diagram]
flowchart LR
    Z["enqueue every node with in-degree 0"] --> P["pop u → append to order"]:::hot
    P --> D["for each edge u → v · in-degree[v]--"]
    D --> Q["in-degree[v] = 0?"]
    Q -->|yes| Z
    P --> E["order size = n? else a cycle exists"]
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Zero in-degree first; peel forward.

> TEMPLATE

Java

```
int[] topoSort(int n, int[][] edges) {
    List<List<Integer>> g = new ArrayList<>();
    int[] indeg = new int[n];
    for (int i = 0; i < n; i++) g.add(new ArrayList<>());
    for (int[] e : edges) { g.get(e[0]).add(e[1]); indeg[e[1]]++; }
    Queue<Integer> q = new LinkedList<>();
    for (int i = 0; i < n; i++) if (indeg[i] == 0) q.add(i); // sources
    int[] order = new int[n]; int k = 0;
    while (!q.isEmpty()) {
        int u = q.poll(); order[k++] = u;
        for (int v : g.get(u)) if (--indeg[v] == 0) q.add(v);
    }
    return k == n ? order : new int[0]; // short output => cycle
}
```

Python

```
from collections import deque
def topo_sort(n, edges):
    g = [[] for _ in range(n)]; indeg = [0]*n
    for a, b in edges:
        g[a].append(b); indeg[b] += 1
    q = deque(i for i in range(n) if indeg[i] == 0) # sources
    order = []
    while q:
        u = q.popleft(); order.append(u)
        for v in g[u]:
            indeg[v] -= 1
            if indeg[v] == 0: q.append(v)
    return order if len(order) == n else [] # cycle -> []
```

C++

```
vector<int> topoSort(int n, vector<vector<int>>& edges) {
    vector<vector<int>> g(n); vector<int> indeg(n, 0);
    for (auto& e : edges) { g[e[0]].push_back(e[1]); indeg[e[1]]++; }
    queue<int> q;
    for (int i = 0; i < n; i++) if (!indeg[i]) q.push(i);
    vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop(); order.push_back(u);
        for (int v : g[u]) if (--indeg[v] == 0) q.push(v);
    }
    return order.size() == n ? order : vector<int>{};
}
```

> COMPLEXITY

Time $O(V + E)$ – build in-degrees, then each edge relaxed once. **Space $O(V + E)$** for the adjacency list and queue.

> PITFALLS

- Forgetting the cycle check (`emitted == n`).
- Reversing edge direction.
- Not seeding *all* zero-in-degree sources.
- Assuming a unique order (many are valid) when the problem demands lexicographically smallest (use a heap).

> CANONICAL PROBLEMS

1. [Course Schedule](#) **Medium**
2. [Course Schedule II](#) **Medium**
3. [Minimum Height Trees](#) **Medium**
4. [Sequence Reconstruction](#) **Medium**
5. [Alien Dictionary](#) **Hard**

// Union-Find

> SIGNAL KEYWORDS

`dynamic connectivity` `are they connected?` `number of groups` `redundant edge` `merge sets`

> WHEN TO USE / NOT USE

Use for near-constant-time grouping and "are A and B connected?" as edges arrive – components, cycle detection in an undirected graph, Kruskal's MST.

Not when you must *remove* edges or recover the actual path between nodes.

> DIAGRAM

```
[diagram]
flowchart LR
    A["find(a) → root ra"] --> Q{"ra = rb?"}
    B["find(b) → root rb"] --> Q
    Q -->|"same root"| C(["already connected"])
    Q -->|"different"| M["attach smaller rank under larger · compress path"]:::hot
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Union by rank; compress the path.

> TEMPLATE

Java

```
int[] parent, rank;
int find(int x) {
    return parent[x] == x ? x : (parent[x] = find(parent[x]));
}
boolean union(int a, int b) {
    int ra = find(a), rb = find(b);
    if (ra == rb) return false; // already together
    if (rank[ra] < rank[rb]) { int t = ra; ra = rb; rb = t; }
    parent[rb] = ra; // attach smaller under larger
    if (rank[ra] == rank[rb]) rank[ra]++;
    return true;
}
```

Python

```

parent, rank = list(range(n)), [0]*n
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]    # path halving
        x = parent[x]
    return x
def union(a, b):
    ra, rb = find(a), find(b)
    if ra == rb: return False
    if rank[ra] < rank[rb]: ra, rb = rb, ra
    parent[rb] = ra
    if rank[ra] == rank[rb]: rank[ra] += 1
    return True

```

C++

```

vector<int> parent, rnk;
int find(int x) { return parent[x]==x ? x : parent[x]=find(parent[x]); }
bool unite(int a, int b) {
    int ra = find(a), rb = find(b);
    if (ra == rb) return false;
    if (rnk[ra] < rnk[rb]) swap(ra, rb);
    parent[rb] = ra;
    if (rnk[ra] == rnk[rb]) rnk[ra]++;
    return true;
}

```

> COMPLEXITY

Time $\sim O(\alpha(n))$ per find/union – inverse Ackermann, effectively constant. **Space** $O(n)$ for parent + rank arrays.

> PITFALLS

- Dropping path compression *or* union-by-rank (degrades to $O(n)$).
- Forgetting `parent[i] = i` init.
- Using it when edges get deleted.
- Component count = number of distinct roots, not n - unions unless you track it.

> CANONICAL PROBLEMS

1. Number of Provinces **Medium**
2. Redundant Connection **Medium**
3. Number of Connected Components in an Undirected Graph **Medium**
4. Accounts Merge **Medium**
5. Making A Large Island **Hard**

// Trie

> SIGNAL KEYWORDS

prefix / starts-with autocomplete dictionary word search grid max XOR

> WHEN TO USE / NOT USE

Use when many strings share prefixes and you query by prefix or whole word – autocomplete, spell-check, grid word search.

Not for single membership checks (a HashSet is simpler) or queries that aren't prefix-shaped.

> DIAGRAM

```
[diagram]
flowchart TD
    R((root)) --> C((c)) --> A((a))
    A --> T(("t ✓")):::hot
    A --> RR((r)) --> E(("e ✓")):::hot
    cap["✓ = a complete word · shared prefixes reuse branches"]
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Branch per character; flag word ends.

> TEMPLATE

Java

```
class Trie {
    Trie[] child = new Trie[26];
    boolean end;
    void insert(String w) {
        Trie node = this;
        for (char c : w.toCharArray()) {
            int i = c - 'a';
            if (node.child[i] == null) node.child[i] = new Trie(); // branch
            node = node.child[i];
        }
        node.end = true; // mark word end
    }
    boolean find(String w, boolean prefix) {
        Trie node = this;
        for (char c : w.toCharArray()) {
            node = node.child[c - 'a'];
            if (node == null) return false;
        }
        return prefix || node.end; // prefix vs full word
    }
}
```

Python

```
class Trie:
    def __init__(self):
        self.child = {}; self.end = False
    def insert(self, w):
        node = self
        for c in w:
            node = node.child.setdefault(c, Trie()) # branch
            node.end = True # mark end
    def find(self, w, prefix):
        node = self
        for c in w:
            if c not in node.child: return False
            node = node.child[c]
        return prefix or node.end
```

C++

```

struct Trie {
    Trie* child[26] = {};
    bool end = false;
    void insert(const string& w) {
        Trie* node = this;
        for (char c : w) {
            int i = c - 'a';
            if (!node->child[i]) node->child[i] = new Trie();
            node = node->child[i];
        }
        node->end = true;
    }
    bool find(const string& w, bool prefix) {
        Trie* node = this;
        for (char c : w) {
            node = node->child[c - 'a'];
            if (!node) return false;
        }
        return prefix || node->end;
    }
};

```

> COMPLEXITY

Time $O(L)$ per insert/search (L = word length), independent of how many words are stored. **Space** $O(\text{alphabet} \times \text{total chars})$ – swap the array for a map when the alphabet is large.

> PITFALLS

- Not marking end (can't tell prefix from a stored word).
- Hard-coding a 26-letter array for unicode/large alphabets.
- Null-check on descent.
- Memory blow-up – a HashMap of children saves space on sparse tries.

> CANONICAL PROBLEMS

1. Implement Trie (Prefix Tree) **Medium**
2. Replace Words **Medium**
3. Design Add and Search Words Data Structure **Medium**
4. Maximum XOR of Two Numbers in an Array **Medium**
5. Word Search II **Hard**

// Greedy

> SIGNAL KEYWORDS

maximize / minimize

interval scheduling

jump / reach

assign / ration

earliest / latest

> WHEN TO USE / NOT USE

Use when a locally optimal choice provably yields the global optimum (justify with an exchange or "stays-ahead" argument) – usually after sorting on the right key.

Not when choices interact and create overlapping subproblems ($\rightarrow$ DP); a greedy guess there is often wrong.

> DIAGRAM

```
[diagram]
flowchart LR
    S["sort by the right key"] --> P["scan left → right"]
    P --> Q{"does the greedy pick fit?"}
    Q -->|yes| T["take it · update state"]::hot
    Q -->|no| K["skip"]
    T --> P
    K --> P
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Take the best local move now.

> TEMPLATE

Java

```
int maxNonOverlap(int[][] iv) {
    Arrays.sort(iv, (a, b) -> Integer.compare(a[1], b[1])); // by end time
    int count = 0, end = Integer.MIN_VALUE;
    for (int[] x : iv) {
        if (x[0] >= end) { // starts after last chosen ends
            count++; end = x[1]; // greedily keep earliest finisher
        }
    }
    return count;
}
```

Python

```
def max_non_overlap(iv):
    iv.sort(key=lambda x: x[1]) # by end time
    count, end = 0, float("-inf")
    for s, e in iv:
        if s >= end: # fits after last chosen
            count += 1; end = e # take earliest finisher
    return count
```

C++

```
int maxNonOverlap(vector<vector<int>>& iv) {
    sort(iv.begin(), iv.end(),
        [](auto& a, auto& b){ return a[1] < b[1]; }); // by end
    int count = 0, end = INT_MIN;
    for (auto& x : iv)
        if (x[0] >= end) { count++; end = x[1]; }
    return count;
}
```

> COMPLEXITY

Time $O(n \log n)$ dominated by the sort (the scan is $O(n)$). **Space** $O(1)$.

> PITFALLS

- Assuming greedy works without proof – many "obvious" greedies are wrong.
- Sorting on the wrong key (end vs start vs ratio).
- Mishandling ties.
- Confusing a greedy problem with one that truly needs DP.

> CANONICAL PROBLEMS

1. Assign Cookies **Easy**
2. Jump Game **Medium**
3. Gas Station **Medium**

4. Task Scheduler Medium

5. Non-overlapping Intervals Medium

// Bit Manipulation

> SIGNAL KEYWORDS

single number / XOR

count bits

power of two

bitmask subsets

toggle / flags

> WHEN TO USE / NOT USE

Use for set membership as a bitmask, parity/XOR cancellation (pairs vanish), low-level flags, or enumerating subsets of $\leq \sim 20$ items.

Not when readability matters more than the micro-optimization – bit tricks are easy to get subtly wrong.

> DIAGRAM

```
[diagram]
flowchart LR
    G["x &gt;&gt; i &amp; 1"] --> RG["read bit i"]
    SB["x | (1 &lt;&lt; i)"] --> RS["set bit i"]:::hot
    CB["x &amp; ~(1 &lt;&lt; i)"] --> RC["clear bit i"]
    XR["x ^ y"] --> RX["toggle · pairs cancel"]:::hot
    LB["x &amp; -x"] --> RL["isolate lowest set bit"]
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Mask, shift, XOR to toggle.

> TEMPLATE

Java

```
boolean getBit(int x, int i) { return (x >> i & 1) == 1; } // read
int setBit(int x, int i)    { return x | (1 << i); }       // set
int clearBit(int x, int i)  { return x & ~(1 << i); }     // clear
int lowestSetBit(int x)     { return x & -x; }            // isolate low 1

int singleNumber(int[] a) { // XOR cancels every pair
    int r = 0;
    for (int v : a) r ^= v;
    return r; // the lone unpaired value
}
```

Python

```
def get_bit(x, i): return (x >> i) & 1 # read
def set_bit(x, i): return x | (1 << i) # set
def clear_bit(x, i): return x & ~(1 << i) # clear
def lowest_set(x): return x & -x # isolate low 1

def single_number(a):
    r = 0
    for v in a: r ^= v # pairs cancel
    return r
```

C++

```

bool getBit(int x, int i) { return (x >> i & 1) == 1; }
int setBit(int x, int i) { return x | (1 << i); }
int clearBit(int x, int i){ return x & ~(1 << i); }
int lowestSetBit(int x) { return x & -x; }

int singleNumber(vector<int>& a) {
    int r = 0;
    for (int v : a) r ^= v;          // pairs cancel
    return r;
}

```

> COMPLEXITY

Time $O(1)$ per bit op; $O(n)$ for a scan like singleNumber. **Space** $O(1)$.

> PITFALLS

- Signed >> vs unsigned >>> in Java.
- $1 << 31$ overflows int (use 1L).
- Operator precedence – always parenthesize masks.
- Off-by-one on bit indices.

> CANONICAL PROBLEMS

1. Number of 1 Bits **Easy**
2. Single Number **Easy**
3. Counting Bits **Easy**
4. Sum of Two Integers **Medium**
5. Single Number II **Medium**

// Dynamic Programming

> SIGNAL KEYWORDS

count the ways min / max cost longest / shortest can you reach target overlapping subproblems

> WHEN TO USE / NOT USE

Use when a problem has *optimal substructure* (answer built from sub-answers) **and** *overlapping subproblems* (same sub-answers recomputed). Define a state, write the recurrence, then memoize or fill a table bottom-up.

Not when a greedy choice is provably optimal (→ Greedy) or subproblems don't repeat (→ plain recursion / divide & conquer).

> DIAGRAM

```

[diagram]
flowchart TD
    P["problem"] --> S["define STATE · minimal info to decide next"]:::hot
    S --> R["write RECURRENCE · state → smaller states"]
    R --> B["set BASE cases"]
    B --> F["fill · memoize (top-down) or table (bottom-up)"]:::hot
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;

```

> MNEMONIC

Mnemonic

Define state, recur, memoize, build up.

> THE SIX RECURRENCES

1. 1-D / Fibonacci – $dp[i]$ from $dp[i-1]$, $dp[i-2]$

Java

```
int rob(int[] a) { // House Robber
    int prev = 0, cur = 0; // dp[i-2], dp[i-1]
    for (int x : a) {
        int take = prev + x; // rob this one
        prev = cur;
        cur = Math.max(cur, take); // best of skip vs rob
    }
    return cur;
}
```

Python

```
def rob(a):
    prev = cur = 0 # dp[i-2], dp[i-1]
    for x in a:
        prev, cur = cur, max(cur, prev + x)
    return cur
```

C++

```
int rob(vector<int>& a) {
    int prev = 0, cur = 0;
    for (int x : a) { int t = prev + x; prev = cur; cur = max(cur, t); }
    return cur;
}
```

Problems: *Climbing Stairs (E)*, *House Robber (M)*.

2. Grid / Kadane – extend or restart a run

```
int maxSubArray(int[] a) { // Kadane
    int best = a[0], cur = a[0];
    for (int i = 1; i < a.length; i++) {
        cur = Math.max(a[i], cur + a[i]); // restart vs extend
        best = Math.max(best, cur);
    }
    return best;
}
```

Problems: *Maximum Subarray (E)*, *Unique Paths (M)*, *Minimum Path Sum (M)*.

3. 0/1 Knapsack – each item once, iterate weight descending

```
int knap01(int[] wt, int[] val, int cap) {
    int[] dp = new int[cap + 1];
    for (int i = 0; i < wt.length; i++)
        for (int w = cap; w >= wt[i]; w--) // descending = use once
            dp[w] = Math.max(dp[w], dp[w - wt[i]] + val[i]);
    return dp[cap];
}
```

Problems: *Partition Equal Subset Sum (M)*, *Target Sum (M)*.

4. Unbounded Knapsack – reuse items, iterate weight ascending

```
int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1); dp[0] = 0;
    for (int c : coins)
        for (int a = c; a <= amount; a++) // ascending = reuse
            dp[a] = Math.min(dp[a], dp[a - c] + 1);
    return dp[amount] > amount ? -1 : dp[amount];
}
```

Problems: *Coin Change (M)*, *Coin Change II (M)*.

5. LCS – 2-D over two sequences

```
int lcs(String s, String t) {
    int m = s.length(), n = t.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++)
            dp[i][j] = s.charAt(i-1) == t.charAt(j-1)
                ? dp[i-1][j-1] + 1 // match -> diagonal + 1
                : Math.max(dp[i-1][j], dp[i][j-1]); // else drop one char
    return dp[m][n];
}
```

Problems: Longest Common Subsequence (M), Edit Distance (H).

6. LIS – patience sorting, $O(n \log n)$

```
int lis(int[] a) {
    List<Integer> tails = new ArrayList<>();
    for (int x : a) {
        int i = Collections.binarySearch(tails, x);
        if (i < 0) i = -(i + 1);
        if (i == tails.size()) tails.add(x); // extend
        else tails.set(i, x); // replace first >= x
    }
    return tails.size();
}
```

Problems: Longest Increasing Subsequence (M), Russian Doll Envelopes (H).

> COMPLEXITY

Time $O(\text{states} \times \text{transitions})$ – e.g. 1-D $O(n)$, LCS $O(m \cdot n)$, knapsack $O(n \cdot \text{cap})$. **Space** $O(\text{states})$, often reducible one dimension (rolling array).

> PITFALLS

- Wrong state (missing info to decide next).
- Wrong iteration order (0/1 needs descending weight, unbounded ascending).
- Missing/incorrect base cases.
- Not memoizing top-down (exponential blowup).
- Off-by-one on the +1 padded table.

> CANONICAL PROBLEMS

1. Climbing Stairs **Easy**
2. Maximum Subarray **Easy**
3. House Robber **Medium**
4. Coin Change **Medium**
5. Longest Common Subsequence **Medium**
6. Longest Increasing Subsequence **Medium**

// Dijkstra

> SIGNAL KEYWORDS

weighted · non-negative min cost to reach shortest path network delay effort / time

> WHEN TO USE / NOT USE

Use for single-source shortest paths on a **non-negative** weighted graph – a min-heap always settles the closest unfinished node first, locking in its distance.

Not with negative edges (→ Bellman-Ford), or on unweighted graphs where plain BFS is simpler and faster.

> DIAGRAM

```
[diagram]
flowchart LR
    I["dist[src]=0, rest=INF · push (src,0)"] --> P["pop nearest node u"]::hot
    P --> S{"stale? d > dist[u]"}
    S -->|yes| P
    S -->|no| R["relax edges: dist[v]=min(dist[v], dist[u]+w)"]
    R --> P
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Greedy settle the nearest unvisited node.

> TEMPLATE

Java

```
int[] dijkstra(List<int[]>[] g, int src, int n) { // g[u] = list of {v, w}
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE); dist[src] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<((a, b) -> a[1] - b[1]);
    pq.offer(new int[]{src, 0});
    while (!pq.isEmpty()) {
        int[] c = pq.poll(); int u = c[0], d = c[1];
        if (d > dist[u]) continue; // skip stale entry
        for (int[] e : g[u])
            if (dist[u] + e[1] < dist[e[0]]) {
                dist[e[0]] = dist[u] + e[1];
                pq.offer(new int[]{e[0], dist[e[0]]});
            }
    }
    return dist;
}
```

Python

```
import heapq
def dijkstra(g, src, n): # g[u] = list of (v, w)
    dist = [float("inf")] * n; dist[src] = 0
    pq = [(0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]: continue # stale
        for v, w in g[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(pq, (dist[v], v))
    return dist
```

C++

```
vector<int> dijkstra(vector<vector<pair<int,int>>>& g, int src, int n) {
    vector<int> dist(n, INT_MAX); dist[src] = 0;
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (auto [v, w] : g[u])
            if (d + w < dist[v]) { dist[v] = d + w; pq.push({dist[v], v}); }
    }
    return dist;
}
```

> COMPLEXITY

Time $O(E \log V)$ – each edge may push once, heap ops are $\log V$. **Space** $O(V + E)$ for the graph and heap.

> PITFALLS

- Negative edges silently break correctness – use Bellman-Ford instead.
- Not skipping stale heap entries ($d > \text{dist}[u]$) – wastes time or re-relaxes.
- Forgetting to initialize dist to infinity (only $\text{src} = 0$).
- Grid problems: the "weight" is often the cell value or the max along the path.

> CANONICAL PROBLEMS

1. Network Delay Time **Medium**
2. Path with Maximum Probability **Medium**
3. Path With Minimum Effort **Medium**
4. Swim in Rising Water **Hard**
5. Minimum Obstacle Removal to Reach Corner **Hard**

// Bellman-Ford

> SIGNAL KEYWORDS

negative weights at most K edges/stops negative cycle cheapest within K relax edges

> WHEN TO USE / NOT USE

Use when edges can be **negative**, when you must bound the path to **at most K edges**, or to **detect a negative cycle** – relax all edges $V-1$ times.

Not on large non-negative graphs where Dijkstra's $O(E \log V)$ beats Bellman-Ford's $O(V \cdot E)$.

> DIAGRAM

```
[diagram]
flowchart LR
    I["dist[src]=0, rest=INF"] --> L["repeat V-1 times (or K+1)"]::hot
    L --> E["for every edge (u,v,w): dist[v]=min(dist[v], dist[u]+w)"]
    E --> L
    L --> C{"one more pass still improves?"}
    C -->|yes| N["negative cycle"]::hot
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Relax every edge V minus once.

> TEMPLATE

Java

```
int cheapestKStops(int n, int[][] edges, int src, int dst, int K) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE); dist[src] = 0;
    for (int i = 0; i <= K; i++) {
        // at most K+1 edges
        int[] tmp = dist.clone(); // freeze last round
        for (int[] e : edges) { // e = {u, v, w}
            if (dist[e[0]] == Integer.MAX_VALUE) continue;
            tmp[e[1]] = Math.min(tmp[e[1]], dist[e[0]] + e[2]);
        }
        dist = tmp;
    }
    return dist[dst] == Integer.MAX_VALUE ? -1 : dist[dst];
}
```

Python

```
def cheapest_k_stops(n, edges, src, dst, K):
    dist = [float("inf")] * n; dist[src] = 0
    for _ in range(K + 1):
        tmp = dist[:]
        for u, v, w in edges:
            if dist[u] != float("inf"):
                tmp[v] = min(tmp[v], dist[u] + w)
        dist = tmp
    return -1 if dist[dst] == float("inf") else dist[dst]
```

C++

```
int cheapestKStops(int n, vector<vector<int>>& edges, int src, int dst, int K) {
    vector<int> dist(n, INT_MAX); dist[src] = 0;
    for (int i = 0; i <= K; i++) {
        vector<int> tmp = dist;
        for (auto& e : edges) {
            if (dist[e[0]] == INT_MAX) continue;
            tmp[e[1]] = min(tmp[e[1]], dist[e[0]] + e[2]);
        }
        dist = tmp;
    }
    return dist[dst] == INT_MAX ? -1 : dist[dst];
}
```

> COMPLEXITY

Time $O(V \cdot E)$ – $V-1$ rounds over all edges. **Space** $O(V)$ for the distance array.

> PITFALLS

- The K-stops variant needs a **frozen copy** each round, else you use more than K edges in one pass.
- A negative cycle is detected when a V -th relaxation still improves a distance.
- Integer overflow when adding to `INT_MAX` – guard with the "unreachable" check.
- Off-by-one on K (K stops means $K+1$ edges).

> CANONICAL PROBLEMS

1. Cheapest Flights Within K Stops **Medium**
2. Network Delay Time **Medium**
3. Shortest Path with Alternating Colors **Medium**
4. Find the City With the Smallest Number of Neighbors **Medium**
5. Minimum Cost to Make at Least One Valid Path in a Grid **Hard**

// Minimum Spanning Tree

> SIGNAL KEYWORDS

connect all at min cost

spanning tree

connect points/cities

min wiring

Kruskal / Prim

> WHEN TO USE / NOT USE

Use to connect **all** vertices with minimum total edge weight – Kruskal (sort edges + Union-Find) or Prim (grow a tree with a heap).

Not for shortest path between two nodes (→ Dijkstra), or directed graphs – MST is for undirected connectivity.

> DIAGRAM

```
[diagram]
flowchart LR
    S["sort edges by weight (ascending)"] --> E["take next cheapest edge"]:::hot
    E --> Q{"union(u,v) forms a cycle?"}
    Q -->|no| T["add edge · cost += w"]:::hot
    Q -->|yes| K["skip it"]
    T --> D{"n-1 edges chosen?"}
    D -->|no| E
    K --> E
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Cheapest edges that avoid a cycle.

> TEMPLATE

Java

```
int mst(int n, int[][] edges) { // edges = {u, v, w}
    Arrays.sort(edges, (a, b) -> Integer.compare(a[2], b[2]));
    int[] parent = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i;
    int cost = 0, used = 0;
    for (int[] e : edges) {
        int ra = find(parent, e[0]), rb = find(parent, e[1]);
        if (ra != rb) { // no cycle -> take edge
            parent[ra] = rb; cost += e[2];
            if (++used == n - 1) break; // tree complete
        }
    }
    return cost;
}
int find(int[] p, int x) { return p[x] == x ? x : (p[x] = find(p, p[x])); }
```

Python

```
def mst(n, edges): # edges = [u, v, w]
    edges.sort(key=lambda e: e[2])
    parent = list(range(n))
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]; x = parent[x]
        return x
    cost = used = 0
    for u, v, w in edges:
        ru, rv = find(u), find(v)
        if ru != rv: # no cycle
            parent[ru] = rv; cost += w; used += 1
            if used == n - 1: break
    return cost
```

C++

```
int find(vector<int>& p, int x){ return p[x]==x ? x : p[x]=find(p,p[x]); }
int mst(int n, vector<vector<int>>& edges) {
    sort(edges.begin(), edges.end(),
        [](auto& a, auto& b){ return a[2] < b[2]; });
    vector<int> p(n); iota(p.begin(), p.end(), 0);
    int cost = 0, used = 0;
    for (auto& e : edges) {
        int ra = find(p, e[0]), rb = find(p, e[1]);
        if (ra != rb) { p[ra] = rb; cost += e[2]; if (++used == n-1) break; }
    }
    return cost;
}
```

> COMPLEXITY

Time $O(E \log E)$ for the sort (Union-Find ops are $\sim O(\alpha)$). **Space** $O(V)$ for the parent array.

> PITFALLS

- Needs Union-Find for cycle checks; comparator overflow on $a[2]-b[2]$ (use `Integer.compare`).
- Stop once $n-1$ edges are chosen.
- A disconnected graph has no spanning tree (used $< n-1$).
- For a complete graph (points), generate all pairwise edges or use Prim to avoid $O(n^2)$ memory.

> CANONICAL PROBLEMS

1. Min Cost to Connect All Points **Medium**
2. Connecting Cities With Minimum Cost **Medium**
3. Optimize Water Distribution in a Village **Hard**
4. Find Critical and Pseudo-Critical Edges in MST **Hard**
5. Checking Existence of Edge Length Limited Paths **Hard**

// Tarjan (SCC & Bridges)

> SIGNAL KEYWORDS

strongly connected components

critical connections

bridges

articulation points

low-link

> WHEN TO USE / NOT USE

Use to find SCCs, bridges, or articulation points in one DFS by comparing each node's **discovery time** to the lowest reachable **low-link** time.

Not for plain connectivity or grouping that only merges ($\rightarrow$ Union-Find), or simple cycle existence ($\rightarrow$ DFS colors / topological sort).

> DIAGRAM

```
[diagram]
flowchart LR
    V["dfs(u): disc[u]=low[u]=++timer"]::hot --> N["for neighbor v"]
    N --> T{"v unvisited?"}
    T -->|yes| R["dfs(v); low[u]=min(low[u],low[v])"]
    R --> B{"low[v] > disc[u]?"}
    B -->|yes| BR["edge u-v is a BRIDGE"]::hot
    T -->|no (back edge)| L["low[u]=min(low[u],disc[v])"]
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Track discovery and low-link times.

> TEMPLATE

Java

```

int timer = 0;
void dfs(int u, int parent, List<Integer>[] g, int[] disc, int[] low,
        List<List<Integer>> bridges) {
    disc[u] = low[u] = ++timer;
    for (int v : g[u]) {
        if (v == parent) continue;
        if (disc[v] == 0) { // tree edge
            dfs(v, u, g, disc, low, bridges);
            low[u] = Math.min(low[u], low[v]);
            if (low[v] > disc[u]) // no back-edge -> bridge
                bridges.add(List.of(u, v));
        } else {
            low[u] = Math.min(low[u], disc[v]); // back edge
        }
    }
}

```

Python

```

def bridges(n, g):
    disc = [0]*n; low = [0]*n; out = []; timer = [0]
    def dfs(u, parent):
        timer[0] += 1; disc[u] = low[u] = timer[0]
        for v in g[u]:
            if v == parent: continue
            if disc[v] == 0: # tree edge
                dfs(v, u)
                low[u] = min(low[u], low[v])
                if low[v] > disc[u]: out.append((u, v))
            else:
                low[u] = min(low[u], disc[v]) # back edge
    for i in range(n):
        if disc[i] == 0: dfs(i, -1)
    return out

```

C++

```

int timer = 0;
void dfs(int u, int parent, vector<vector<int>>& g, vector<int>& disc,
        vector<int>& low, vector<vector<int>>& bridges) {
    disc[u] = low[u] = ++timer;
    for (int v : g[u]) {
        if (v == parent) continue;
        if (!disc[v]) { // tree edge
            dfs(v, u, g, disc, low, bridges);
            low[u] = min(low[u], low[v]);
            if (low[v] > disc[u]) bridges.push_back({u, v});
        } else low[u] = min(low[u], disc[v]); // back edge
    }
}

```

> COMPLEXITY

Time $O(V + E)$ – a single DFS. **Space** $O(V + E)$ for the graph plus disc/low arrays and recursion.

> PITFALLS

- Skip the parent edge only **once** (careful with multi-edges).
- `disc == 0` doubles as the "unvisited" sentinel – timer starts at 1.
- Bridge test uses `low[v] > disc[u]`; articulation points use `low[v] >= disc[u]` (plus a root special case).
- Deep recursion can overflow the stack on large graphs – consider an iterative DFS.

> CANONICAL PROBLEMS

1. Critical Connections in a Network **Hard**
2. Find Eventual Safe States **Medium**

3. Course Schedule II **Medium**
4. Longest Cycle in a Graph **Hard**
5. Number of Good Paths **Hard**

// String Matching

> SIGNAL KEYWORDS

pattern in text needle in haystack repeated substring prefix = suffix KMP / Rabin-Karp

> WHEN TO USE / NOT USE

Use to locate a pattern in a text in $O(n+m)$ – KMP builds a failure table so the text pointer never moves backward; Rabin-Karp rolls a hash across windows.

Not for approximate / fuzzy matching or multi-pattern search (→ Aho-Corasick, edit distance) – this is exact single-pattern matching.

> DIAGRAM

```
[diagram]
flowchart LR
    B["build lps[]: longest proper prefix = suffix"]::hot --> S["scan text with pointer i, pattern pointer j"]
    S --> M["text[i] == pat[j]?"]
    M -->|yes| A["i++, j++ · j==m ⇒ match"]::hot
    M -->|no| F["j = lps[j-1] (jump, don't rewind i)"]
    F --> S
    A --> S
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Precompute prefix jumps; never backtrack.

> TEMPLATE

Java

```
int[] buildLps(String p) { // lps[i] = longest prefix==suffix of p[0..i]
    int[] lps = new int[p.length()];
    int len = 0;
    for (int i = 1; i < p.length(); ) {
        if (p.charAt(i) == p.charAt(len)) lps[i++] = ++len;
        else if (len > 0) len = lps[len - 1]; // fall back
        else lps[i++] = 0;
    }
    return lps;
}

int kmp(String t, String p) {
    int[] lps = buildLps(p);
    for (int i = 0, j = 0; i < t.length(); ) {
        if (t.charAt(i) == p.charAt(j)) { i++; j++; if (j == p.length()) return i - j; }
        else if (j > 0) j = lps[j - 1];
        else i++;
    }
    return -1;
}
```

Python

```
def build_lps(p):
    lps = [0] * len(p); length = 0; i = 1
    while i < len(p):
        if p[i] == p[length]: length += 1; lps[i] = length; i += 1
        elif length: length = lps[length - 1] # fall back
        else: lps[i] = 0; i += 1
    return lps
def kmp(t, p):
    lps = build_lps(p); i = j = 0
    while i < len(t):
        if t[i] == p[j]:
            i += 1; j += 1
            if j == len(p): return i - j
        elif j: j = lps[j - 1]
        else: i += 1
    return -1
```

C++

```
vector<int> buildLps(const string& p) {
    vector<int> lps(p.size()); int len = 0;
    for (int i = 1; i < p.size(); ) {
        if (p[i] == p[len]) lps[i++] = ++len;
        else if (len) len = lps[len - 1];
        else lps[i++] = 0;
    }
    return lps;
}
```

> COMPLEXITY

Time $O(n + m)$ – build the table in $O(m)$, scan the text in $O(n)$. **Space** $O(m)$ for the lps array.

> PITFALLS

- On mismatch, fall back to `lps[len-1]`, not to 0 – that's what preserves linearity.
- Off-by-one when returning the match index (`i - j`).
- Rabin-Karp must **verify** a hash hit character-by-character (collisions) and use modular arithmetic to avoid overflow.
- Building lps compares within the pattern only, never the text.

> CANONICAL PROBLEMS

1. Find the Index of the First Occurrence in a String **Easy**
2. Repeated Substring Pattern **Easy**
3. Repeated String Match **Medium**
4. Shortest Palindrome **Hard**
5. Longest Happy Prefix **Hard**

// Interval DP

> SIGNAL KEYWORDS

range `[i, j]` from splits

burst balloons

merge stones

cut a stick

matrix chain

> WHEN TO USE / NOT USE

Use when the answer for an interval `[i, j]` is built from a **split point k** inside it (`dp[i][k] + dp[k+1][j] + cost`). Fill by **increasing interval length**.

Not for linear/1-D DP where state only depends on a prefix ($\rightarrow$ standard DP), or when there's no "combine two sub-ranges" structure.

> DIAGRAM

```
[diagram]
flowchart LR
    L["for len = 2 .. n"]:::hot --> I["for each start i, j = i+len-1"]
    I --> K["try every split k in [i, j)"]
    K --> C["dp[i][j] = best over k of dp[i][k] + dp[k+1][j] + cost"]:::hot
    C --> I
    I --> L
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
```

> MNEMONIC

Mnemonic

Solve inner intervals; merge outward.

> TEMPLATE

Java

```
int intervalDP(int[] a) {
    int n = a.length;
    int[][] dp = new int[n][n];           // dp[i][j] = best on [i, j]
    for (int len = 2; len <= n; len++)    // grow interval length
        for (int i = 0; i + len - 1 < n; i++) {
            int j = i + len - 1;
            dp[i][j] = Integer.MAX_VALUE;
            for (int k = i; k < j; k++)    // split point
                dp[i][j] = Math.min(dp[i][j],
                                     dp[i][k] + dp[k + 1][j] + cost(a, i, j, k));
        }
    return dp[0][n - 1];
}
```

Python

```
def interval_dp(a):
    n = len(a)
    dp = [[0] * n for _ in range(n)]      # dp[i][j] = best on [i, j]
    for length in range(2, n + 1):        # grow length
        for i in range(0, n - length + 1):
            j = i + length - 1
            dp[i][j] = min(dp[i][k] + dp[k + 1][j] + cost(a, i, j, k)
                           for k in range(i, j))
    return dp[0][n - 1]
```

C++

```
int intervalDP(vector<int>& a) {
    int n = a.size();
    vector<vector<int>> dp(n, vector<int>(n, 0));
    for (int len = 2; len <= n; len++)
        for (int i = 0; i + len - 1 < n; i++) {
            int j = i + len - 1; dp[i][j] = INT_MAX;
            for (int k = i; k < j; k++)
                dp[i][j] = min(dp[i][j], dp[i][k] + dp[k+1][j] + cost(a, i, j, k));
        }
    return dp[0][n - 1];
}
```

> COMPLEXITY

Time $O(n^3)$ – $O(n^2)$ intervals $\times$ $O(n)$ split points. **Space** $O(n^2)$ for the table.

> PITFALLS

- Iterate by **length**, not raw i, j – sub-intervals must be solved first.
- Base case is length 1 (often 0); initialize the rest carefully (MIN vs MAX).
- The k range is $[i, j)$; off-by-one here is the classic bug.

- Burst Balloons reverses the intuition: think of the **last** balloon popped in [i, j].

> CANONICAL PROBLEMS

1. Longest Palindromic Subsequence **Medium**
2. Stone Game **Medium**
3. Minimum Cost to Cut a Stick **Hard**
4. Burst Balloons **Hard**
5. Palindrome Partitioning II **Hard**

// LRU / LFU Cache Design

> SIGNAL KEYWORDS

design a cache

O(1) get / put

least recently used

least frequently used

fixed capacity

> WHEN TO USE / NOT USE

Use to get **O(1)** get/put with eviction: a **hashmap** (key → node) for lookup plus a **doubly linked list** for recency order. LFU adds per-frequency buckets.

Not when there's no eviction / ordering requirement – a plain HashMap already does O(1) lookups.

> DIAGRAM

```
[diagram]
flowchart LR
    M["M[hashmap: key → node]:::hot --> D[doubly linked list: MRU ... LRU]"]
    G["G[get(k): move node to front (MRU)]:::hot --> D"]
    P["P[put(k): insert at front · if full, drop tail (LRU)] --> D"]
    classDef hot fill:#3a1418,stroke:#ff5c5c,color:#ffffff,stroke-width:2px;
    class M,G,P hot;
```

> MNEMONIC

Mnemonic

Hashmap plus linked list equals O(1).

> TEMPLATE

Java

```
class LRUCache extends LinkedHashMap<Integer, Integer> {
    private final int cap;
    LRUCache(int cap) {
        super(cap, 0.75f, true);          // true = access-order
        this.cap = cap;
    }
    public int get(int key) { return super.getOrDefault(key, -1); }
    public void put(int key, int value) { super.put(key, value); }
    protected boolean removeEldestEntry(Map.Entry<Integer, Integer> e) {
        return size() > cap;              // evict LRU when over capacity
    }
}
```

Python

```

from collections import OrderedDict
class LRUCache:
    def __init__(self, cap):
        self.cache = OrderedDict(); self.cap = cap
    def get(self, key):
        if key not in self.cache: return -1
        self.cache.move_to_end(key)      # mark most-recent
        return self.cache[key]
    def put(self, key, value):
        if key in self.cache: self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.cap:
            self.cache.popitem(last=False)  # evict LRU

```

C++

```

class LRUCache {
    int cap; list<pair<int,int>> dll;           // front = MRU
    unordered_map<int, list<pair<int,int>>::iterator> mp;
public:
    LRUCache(int c) : cap(c) {}
    int get(int key) {
        if (!mp.count(key)) return -1;
        dll.splice(dll.begin(), dll, mp[key]); // move to front
        return mp[key]->second;
    }
    void put(int key, int val) {
        if (mp.count(key)) dll.erase(mp[key]);
        dll.push_front({key, val}); mp[key] = dll.begin();
        if (dll.size() > cap) { mp.erase(dll.back().first); dll.pop_back(); }
    }
};

```

> COMPLEXITY

Time $O(1)$ for get and put (hashmap lookup + constant list splice). **Space** $O(\text{capacity})$.

> PITFALLS

- LRU with LinkedHashMap needs the **access-order** flag (true), else it tracks insertion order.
- A hand-rolled doubly linked list needs **dummy head/tail** to avoid null checks.
- Update recency on **every** get, not just put.
- LFU is harder: keep a map of frequency → ordered list and track the current minimum frequency for $O(1)$ eviction.

> CANONICAL PROBLEMS

1. Design HashMap **Easy**
2. LRU Cache **Medium**
3. Design Browser History **Medium**
4. LFU Cache **Hard**
5. All O`one Data Structure **Hard**